Laboratorio 4 – Problema E

Burbuja de inclinación con ATmega328P, MPU6050 y matriz WS2812B 8×8

Mini informe técnico (borrador para el informe completo)

27 de octubre de 2026

1. Objetivo

Construir un nivel electrónico ("burbuja") con un ATmega328P a 16 MHz. El acelerómetro MPU6050 se lee por I2C, se calculan los ángulos de inclinación (*roll* y *pitch*) y se muestra una burbuja como un LED en una matriz RGB WS2812B de 8×8. El sistema incluye una función HOLD, calibración guardada en EEPROM, aviso sonoro al llegar al borde de la matriz y salida por UART a 38400 baud (8N1).

2. Hardware y conexiones

Pin Arduino	**ATmega328P**	**Función**
A4	PC4	SDA (I2C con el MPU6050)
A5	PC5	SCL (I2C con el MPU6050)
D1	PD1	TX de la UART (monitor serie)
D2	PD2	Pulsador HOLD (a GND, pull-up interno)
D3	PD3	Pulsador CALIBRAR (a GND, pull-up interno)
D6	PD6	Datos de la matriz WS2812B
D8	PB0	Buzzer

Tabla 1: Conexiones del proyecto.

El MPU6050 tiene dirección I2C `0x68` (AD0 a GND) y el bus trabaja a 100 kHz. La matriz necesita alimentación de 5 V adecuada para 64 LEDs, una resistencia de 330 Ω en serie con la línea de datos, un capacitor de reserva entre 5 V y GND, y GND común con el Arduino.

3. Funcionamiento

3.1. Lectura del sensor

Al iniciar se verifica el registro `WHO_AM_I` (`0x75`, debe dar `0x68`), se despierta el sensor y se configura: filtro interno de ≈44 Hz, muestreo a 50 Hz, acelerómetro en ±2 g y giroscopio en ±250 °/s. El programa lee 14 bytes desde `0x3B` cada 20 ms. La comunicación I2C está hecha con los registros TWBR/TWCR/TWDR, con *timeout* para que un fallo no cuelgue el programa.

3.2. Cálculo de ángulos

A partir de las aceleraciones a_x, a_y, a_z:

$$\text{roll} = \text{atan2}(a_y,\, a_z) \cdot \frac{180}{\pi} \tag{1}$$

$$\text{pitch} = \text{atan2}\Big(-a_x,\, \sqrt{a_y^2 + a_z^2}\Big) \cdot \frac{180}{\pi} \tag{2}$$

A cada ángulo se le resta el *offset* de calibración, y luego se aplica una media móvil de 8 muestras para estabilizar la imagen.

3.3. De ángulo a posición del LED

Cada eje se convierte a una coordenada de 0 a 7. El centro es el LED (4,4):

- Zona muerta: si $|\theta| \leq 3°$, la coordenada es 4.
- Saturación: $\theta \leq -30° \Rightarrow 0$ y $\theta \geq 30° \Rightarrow 7$.
- Entre medio: interpolación lineal, $c = \text{round}\big((\theta + 30) \cdot 7/60\big)$.

El roll define la columna y el pitch define la fila.

3.4. Color de la burbuja

Color	**Condición**
Azul	Nivelado: roll y pitch dentro de la zona muerta ($\leq 3°$)
Rojo	Predomina la inclinación en roll (eje X)
Verde	Predomina la inclinación en pitch (eje Y)
Amarillo	Ambos ejes fuera de zona muerta y de magnitud similar (relación menor a 2:1)

Tabla 2: Código de colores de la burbuja.

3.5. Aviso de borde

Si la burbuja llega a la fila o columna 0 o 7, el buzzer suena una vez durante 100 ms al entrar al borde y el LED parpadea cada 250 ms mientras permanezca allí.

3.6. Matriz WS2812B

Cada LED recibe 24 bits (orden GRB, MSB primero) con un periodo de 1,25 μs por bit. Como los tiempos son críticos, la imagen se arma en C y la transmisión de cada bit está en ensamblador en línea (20 ciclos por bit a 16 MHz). Esta rutina solo es válida a 16 MHz. El brillo se limita a 24 para reducir el consumo.

4. Máquina de estados

El programa usa cinco estados (Figura 1).

Estado	Descripción
INICIO	Configura el MPU6050. Si responde pasa a MEDIR; si no, a ERROR_SENSOR.
MEDIR	Muestrea cada 20 ms, filtra, actualiza la burbuja, maneja el aviso de borde y envía datos por UART.
HOLD	Congela la imagen; el sensor sigue leyéndose y el monitor serie sigue activo. Se suspenden el pitido y el parpadeo.
CALIBRAR	Promedia 32 lecturas con el sensor quieto y guarda los *offsets*.
ERROR_SENSOR	Ante una lectura inválida o falla de I2C: LED rojo central intermitente, buzzer apagado, mensaje por UART y reintento tras 1 s.

Tabla 3: Estados del programa.

Botón 1 alterna MEDIR/HOLD. Botón 2 inicia la calibración y tiene prioridad si se pulsan ambos. Los pulsadores usan antirrebote por software de 30 ms.

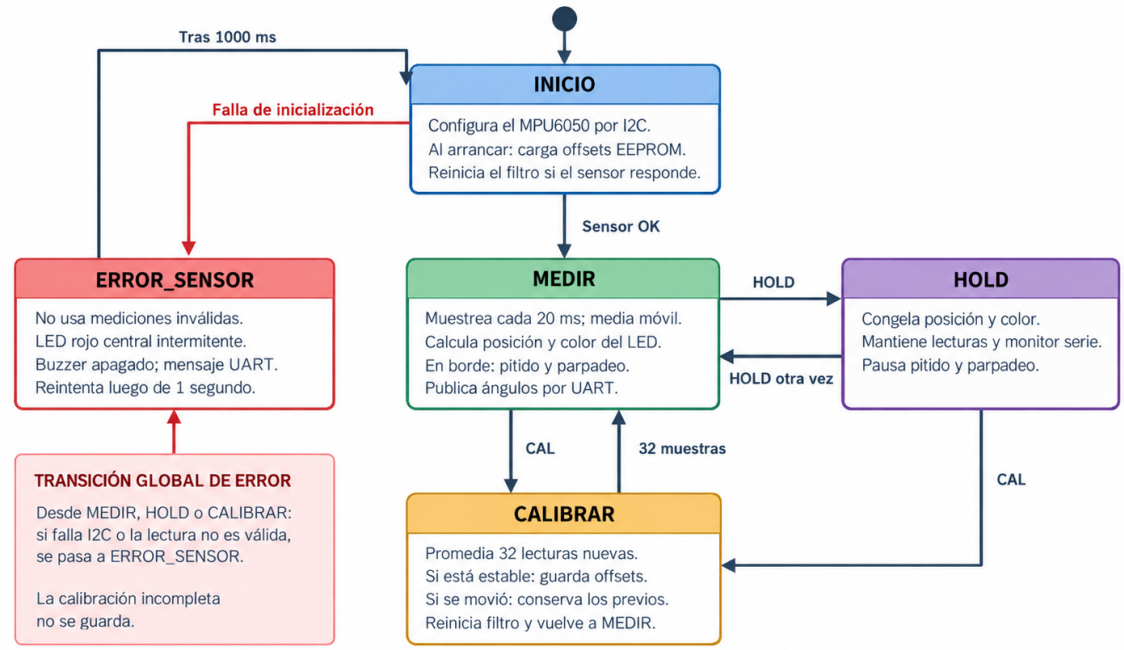


Figura 1: Diagrama de la máquina de estados (diseño del programa, no es una captura de ejecución).

5. Calibración y EEPROM

Al pulsar CALIBRAR con el sensor quieto, se toman 32 muestras (≈0,64 s). Si la dispersión de cada ángulo es $\leq 1{,}5°$, el promedio se guarda como *offset*; si no, se conservan los anteriores y se informa por UART ("movimiento; repetir"). Los *offsets* se guardan en centésimas de grado con firma (`0xA5`) y *checksum* (XOR con `0x5A`). La firma se invalida antes de escribir y se valida al final, de modo que un corte de energía no deja una calibración incompleta como válida. Al encender, los *offsets* se cargan automáticamente.

6. Salida por UART

A 38400 baud (UBRR = 25), se envía cada 250 ms una línea como:

```
R=0.0 P=0.0 F=4 C=4 E=1 COLOR=0
```

donde R = roll, P = pitch, F = fila, C = columna, E = estado y COLOR el código de color (0 azul, 1 rojo, 2 verde, 3 amarillo). Los estados se numeran 0 INICIO, 1 MEDIR, 2 HOLD, 3 CALIBRAR, 4 ERROR_SENSOR.

7. Estructura del software

El proyecto se entrega como un único `main.c` para Microchip Studio, que reúne estos módulos:

Módulo	Responsabilidad
UART	Inicialización y envío de texto/números (sin printf de punto flotante).
I2C	Protocolo TWI con *timeout* y verificación de ACK.
MPU6050	Configuración, lectura de aceleración/giro y cálculo de ángulos.
TIEMPO	Reloj en milisegundos con Timer1 (prescaler 64, 4 μs por cuenta).
NIVEL	Media móvil, zona muerta, coordenadas y color.
MATRIZ	Transmisión de píxeles a la WS2812B.
MEMORIA	Lectura/escritura de *offsets* en EEPROM con firma y *checksum*.
BOTONES	Lectura con antirrebote; genera un evento por pulsación.

Tabla 4: Módulos del programa.

Fragmento clave: conversión de ángulo a coordenada.

```
uint8_t nivel_coordenada(float angulo) {
    if (fabsf(angulo) <= 3.0f) return 4; /* zona muerta */
    if (angulo <= -30.0f) return 0;
    if (angulo >= 30.0f) return 7;
    return (uint8_t)(((angulo + 30.0f) * 7.0f / 60.0f) + 0.5f);
}
```

8. Compilación y simulación

Compilación. Proyecto *GCC C Executable* para ATmega328P en Microchip Studio, optimización `-Os`, biblioteca `m` agregada en el enlazador, `F_CPU = 16000000UL` y compilación con F7.

Simulación. PICSimLab 0.9.2 (Arduino Uno, ATmega328P, 16 MHz, terminal a 38400 baud) con MPU6050, matriz WS2812B, dos pulsadores y buzzer en *Spare Parts*. El MPU6050 simulado se maneja con los controles de aceleración. Casos de prueba definidos en el repositorio:

- Nivelado: $a_x = 0,\ a_y = 0,\ a_z = 1$ g $\Rightarrow$ punto azul en (4,4).
- Roll $\approx +20°$: $a_x = 0,\ a_y = 0{,}342,\ a_z = 0{,}940$ g.
- Pitch $\approx +20°$: $a_x = -0{,}342,\ a_y = 0,\ a_z = 0{,}940$ g.

(Giroscopio en cero, porque las pruebas son estáticas.)

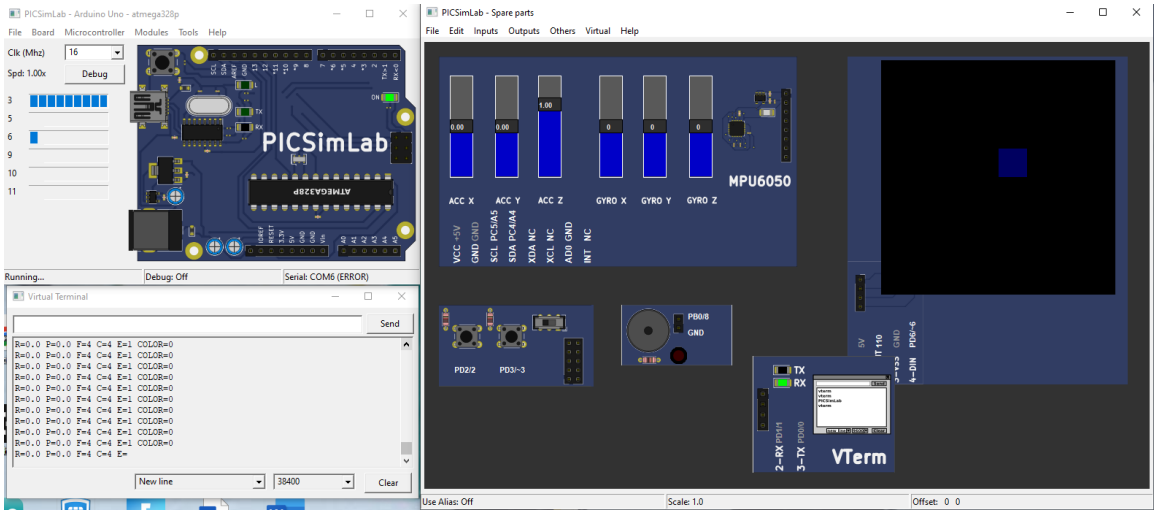


Figura 2: Simulación en PICSimLab: sensor nivelado ($a_z = 1$ g), estado MEDIR, ángulos 0,0 y punto azul en (4,4).

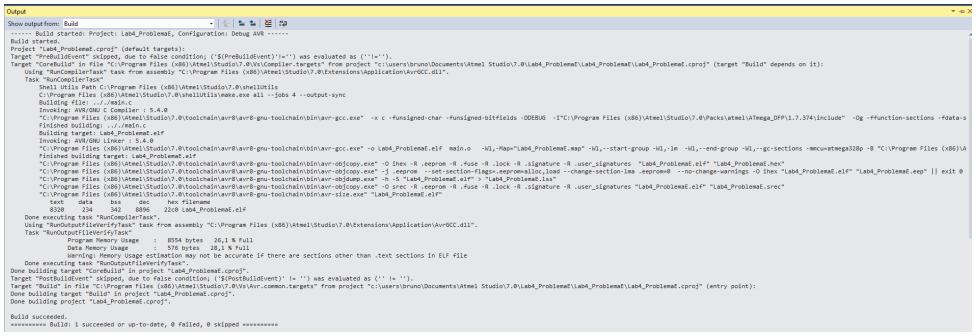


Figura 3: Compilación en Microchip Studio (configuración Debug con `-Og`): programa de 8554 bytes y 576 bytes de RAM, *Build succeeded*.

9. Resultados de las pruebas

Se probaron en simulación (PICSimLab) todas las funciones del programa y funcionaron correctamente.

Prueba	**Resultado**
Compilación sin errores	Correcta.
Lectura del MPU6050 por I2C y salida por UART	Funciona correctamente.
Posición y color de la burbuja (nivelado, roll, pitch y combinado)	Funciona correctamente.
Zona muerta, saturación y aviso de borde (buzzer y parpadeo)	Funciona correctamente.
HOLD	Funciona correctamente.
Calibración y EEPROM (se conserva al reiniciar)	Funciona correctamente.
Error de sensor	Funciona correctamente.

Tabla 5: Resultados de las pruebas en simulación.